No Permanent Champion: A Deployment Study of SGLang and vLLM across Dense, Ultra-Sparse MoE, and KDA-Hybrid Architectures on H200 GPUs

Chao-Chun Chuang[1,*], Po-Hsiang Lin[2], and Liang-Hsun Huang[3]

[1]National Center for High-performance Computing (NCHC),
National Institutes of Applied Research (NIAR), Taiwan
[2]Kaohsiung Veterans General Hospital, Kaohsiung, Taiwan
[3]Twinkle AI, Taiwan

∗ Corresponding author: c00cjz00@nchc.org.tw

September 2026

Abstract

We operate a multi-model API service on an HPC cluster—a LiteLLM gateway fronting Slurm-managed SGLang and vLLM engines on NVIDIA H200 GPUs—and each new model it onboards raises the same question: which serving framework will run this checkpoint faster? We encoded the comparison as versioned procedures ("skills") that an LLM-based agent executes under human approval: both frameworks' engines are scaffolded from official per-model documentation, deployed side by side under traffic-isolated aliases, and driven with the same 500–1,000-request burst in five repetitions under an interleaved, warmed-up protocol (with one documented rerun exception). Across three 2026 models—a 27B dense hybrid, a 180B-total/6B-active ultra-sparse MoE, and a 288-expert Kimi Delta Attention (KDA) hybrid MoE—the leader depended on the architecture: SGLang led the dense model by 9.6% on average and in every paired repetition, while vLLM led the MoE by 2.40× and the KDA hybrid by 28% (30% across four independent engine-instance pairs). In each framework's deployable configuration, speculative decoding with the checkpoints' integrated draft heads lowered throughput in five of six framework–model pairs, by up to 49%. For SGLang on the dense model the loss came from memory rather than computation: with EAGLE's footprint, 80 was the highest concurrency cap that started in our sweep, whereas at equal concurrency EAGLE raised throughput by 15%. An earlier single run without warm-up had reversed the KDA ranking, understating vLLM by 39%; warm-up and repetition recovered it. An agent-assisted layered diagnosis also raised mixed 1,500-user success from 67.9% to 100% by correcting file-descriptor limits. Framework choice and speculative decoding should therefore be measured per model, with repetition, rather than inherited as defaults; the procedure and the raw measurements behind the comparisons are released.

1 Introduction

The throughput and latency of a deployed large language model (LLM) depend not only on the model and the accelerator, but on the serving framework that schedules batches, manages KV-cache memory, and executes attention and mixture-of-experts (MoE) kernels. Among open-source serving frameworks, a study of open-source software systems found vLLM to be the most visible in popularity and adoption, with SGLang among its principal alternatives [1]. vLLM was introduced with PagedAttention for efficient KV-cache memory management [2], and SGLang

with a RadixAttention prefix cache and a structured-generation runtime [3]. Both systems evolve quickly, add support for new model architectures as they appear, and make different implementation trade-offs for the same architecture. Practitioners therefore face a recurring question that vendor documentation does not answer: *for this specific model, on this hardware, which framework will serve it faster?*

This question is not academic for us. We operate a multi-model API service for the users of an HPC cluster: a LiteLLM gateway fronting Slurm-managed inference engines—SGLang and vLLM containers among them—on NVIDIA H200 GPUs, with per-engine lifecycle management, versioned container images, and declarative routing that lets a candidate engine receive production-shaped traffic under isolated aliases before it claims any production name. Every time the center onboards a new open-weight model, the platform forces a concrete framework decision, and the wrong one is expensive: as this paper shows, the cost of choosing wrongly reaches 2.4× in throughput—larger than any tuning lever we measured, including correcting an outright misconfiguration (+48% on the dense engine's earlier settings). Manual evaluation is slow and error-prone (framework-specific flags, memory budgets, and validation steps differ), so we delegated it to an agent: the evaluation procedure—engine scaffolding from each framework's official per-model documentation, dual-framework deployment, paired benchmarking, verdict recording, and production traffic switching—was encoded as declarative, versioned procedures (which we call *skills*) and executed by an LLM-based coding agent under human approval.

Published comparisons between serving systems do exist: an empirical evaluation of vLLM against HuggingFace TGI on LLaMA-2 models is a recent example [4], surveys consolidate the serving-systems literature [5], and a study of open-source deployments documents which frameworks practitioners actually adopt [1]. Same-checkpoint comparisons of multiple backends have quantified accuracy fidelity [6], and SiliconBench evaluates nine Apple Silicon serving engines on unified-memory desktops, using CUDA vLLM and SGLang as references [7]; neither addresses throughput at hundreds to a thousand concurrent requests on HPC GPUs, and none evaluates ultra-sparse MoE or KDA hybrids. Separately, the fragility of speculative decoding under batched serving is not merely a vendor warning: goodput analyses and adaptive speculation control have formalized when drafting helps and when it hurts [8, 9]. What remains unmeasured is how these framework and speculation choices behave on the newest architectures—with checkpoint-integrated multi-token prediction (MTP) heads rather than external draft models—under high-concurrency burst load.

The present paper reports what this agent-assisted workflow found, together with the workflow itself. We make five contributions:

1. **An agent-assisted evaluation workflow** (Sections 3–4): a paired A/B procedure—both frameworks serving the *same local checkpoint* on the same GPU type under the *same client workload*, with engine parameters taken from official documentation, five repetitions under an interleaved, warmed-up protocol (one documented exception; Section 4), and every run's statistics recorded—encoded as versioned skills that an LLM-based agent executes under human approval, from scaffolding the candidate engine to switching production traffic.

2. **Three per-model results** (Section 5): SGLang 0.5.20 led by 9.6% on a dense 27B hybrid; vLLM 0.29.1rc1 led by 2.40× on a 180B/6B-active ultra-sparse MoE; and a dedicated vLLM 0.28.1rc1 build led by 28% on a 306 GB FP8 KDA model—so the leading framework is model-specific, and the gap can outweigh every other tuning lever we measured.

3. **Speculative decoding under load** (Sections 5.1–5.4): in each framework's deployable configuration, the checkpoints' integrated draft heads lowered throughput in five of six framework–model pairs (by 7–49%). A cap-48 control and a feasibility sweep show that for

SGLang on the dense model the loss is a memory effect: EAGLE raises throughput by 15% at equal concurrency, but the highest tested feasible cap is 80, where it still trails the plain configuration by 34%.

4. **A measurement-protocol finding** (Sections 5.3 and 6): a single run without warm-up reversed the KDA ranking, understating vLLM by 39%; with warm-up, all four independent engine-instance pairs favored vLLM, with cross-instance CVs of 1–2%.

5. **A production platform description and reliability analysis** (Sections 3 and 5.5): a LiteLLM-based multi-engine gateway with Slurm-managed engine lifecycle, declarative production/staging traffic switching, and versioned container images, together with a layered diagnosis—also agent-assisted, from an encoded symptom-to-cause procedure—that traced mixed-load failures (67.9% → 91.8% → 100%) to file-descriptor limits in the client shell and per-connection sshd, not to GPU engines or the gateway.

All experiments were run in September 2026 on the platform described in Section 3; raw per-run and per-request data are released in the artifact repository (see Data and Artifact Availability).

2 Related Work

Serving systems. Iteration-level scheduling and continuous batching were introduced by Orca [10] and adopted by vLLM, whose PagedAttention brought operating-system-style page management to KV-cache memory [2]. SGLang contributed RadixAttention for automatic prefix reuse and an efficient runtime for structured generation programs [3]. Chunked prefill, proposed in SARATHI, mitigates the interference of long prefills with decodes [11], and DistServe shows that disaggregating prefill from decoding improves goodput under latency constraints [12]. Kernel libraries such as FlashInfer supply portable attention and decoding kernels that both frameworks integrate [13]. Our work does not propose a new serving mechanism; it measures two mature frameworks on a production platform and finds that their relative standing differs across the three models tested.

Model architectures under test. The three models we evaluate span the current design space of efficient open-weight LLMs. Qwen3.8-27B continues the dense hybrid line of Qwen3 [14], combining gated delta net (GDN) linear attention with full attention. Qwen3.8-Flash-Next is an ultra-sparse MoE (180B total, 6B active parameters, 512 experts) with a large N-gram embedding table, in the lineage of DeepSeek-V2/V3's economical MoE and multi-head latent attention (MLA) designs [15, 16]. GLM-5.3-Flash continues the GLM agentic line [17] as a native-FP8, 288-expert MoE (8 experts active per token, one shared expert, first three layers dense) whose 45-layer attention stack interleaves, three-to-one, 34 Kimi Delta Attention (KDA) linear-attention layers—the architecture introduced by Kimi Linear [18]—with 11 DeepSeek Sparse Attention (DSA) layers, plus a 1M-token context window; its closest published relatives include Kimi K2's large sparse-MoE design [19], and serving-side research on KDA-style hybrids is already emerging—recurrent-state compression (DASC) [20], decay-aware mixed-precision state quantization (DAMP) [21], and switchable per-layer mixer placements, including KDA, served from one supernet checkpoint (Super Apriel) [22]. The architectural diversity of this trio is a plausible reason framework rankings might not transfer across models—a hypothesis our results support but cannot isolate, given one model per architecture.

Speculative decoding. Speculative sampling accelerates autoregressive decoding by drafting multiple tokens with a cheap proposal mechanism and verifying them in parallel with the target model [23]. Proposal mechanisms include additional decoding heads (Medusa [24]), feature-level draft models with uncertainty handling (EAGLE [25]), and the checkpoint-integrated MTP heads of

DeepSeek-V3 [15] and its successors, which both frameworks in this study expose as NEXTN/MTP options. The published literature evaluates speculative decoding primarily as a *latency* optimization at low request concurrency [23–25], but its behavior under batched serving has been studied: goodput analyses observe that speculation can degrade serving performance when applied without regard to load, and adaptive systems such as TurboSpec close the loop by enabling drafting only when it pays [8]; AdaServe customizes speculation to per-request SLOs [9]. Our contribution is narrower and complementary: a repeated measurement of *checkpoint-integrated* MTP/NEXTN heads across two frameworks and three architectures under high-concurrency burst load, with controls that distinguish the throughput lost to a reduced concurrency cap from the net effect of speculation at a fixed cap.

Empirical comparisons of serving frameworks. A comparative performance study of vLLM and HuggingFace TGI on LLaMA-2 models (7B–70B) found framework-dependent trade-offs in throughput, latency, and memory [4]; a survey consolidates the broader serving-systems literature [5]; and a large-scale analysis of open-source systems documents which serving frameworks and methods are adopted in practice [1]. The Silent Hyperparameter holds weights and decoding fixed across five backends but measures accuracy reproducibility rather than throughput [6]; SiliconBench evaluates nine Apple Silicon serving engines on unified-memory desktop hardware, using CUDA vLLM and SGLang as references, with fidelity as a first-class metric [7]; and AutoTuneBench has agent systems auto-tune both engines under a strict measurement protocol (paired seeds, cross-run CV at most 5%) [26]. Relative to these, our study differs in target (high-concurrency burst throughput on a production HPC platform with H200 GPUs), in scope (three 2026 hybrid architectures, including—to our knowledge—the first cross-framework serving comparison of a KDA-hybrid model [18]), and in the object of automation: prior agent work tunes engine configurations [26] or generates bespoke serving stacks from scratch [27], whereas our skills guide an agent through the production lifecycle of an existing platform, from deployment behind a live gateway to production traffic switching.

3 The Platform: A Production LLM Serving Stack for HPC Clusters

All experiments ran on the serving platform we operate for internal users of an HPC cluster (NCHC). This section documents the platform both to establish experimental validity (Section 4) and because the platform design itself—engine lifecycle, traffic switching, and image governance—encodes operational lessons that we believe are reusable. Figure 1 outlines the request path; Table 1 summarizes the components.

Three design decisions deserve emphasis because they made the A/B protocol of Section 4 cheap to run and safe for production traffic. *First*, engines are self-contained directories (`engines/<name>/`) whose `config.env` declares the container image, model path, port, GPU topology, concurrency cap, speculative-decoding settings, and alias claims; starting, stopping, and validating an engine is a single command each, and each engine runs as a separate Slurm job, providing job-level resource isolation. *Second*, the gateway's routing configuration is generated from the *live* set of engine endpoints, so bringing a candidate engine up under suffixed aliases lets it receive production-shaped traffic for validation without claiming the production names; promoting it (or rolling back) is a two-line configuration edit and a ten-second gateway restart. *Third*, container images are version-named SIF files that the pull script refuses to overwrite by default, with a policy of using a vendor's *dedicated* build when one exists for a given model; this policy was validated during the study when a generic nightly vLLM build failed on the KDA model (kernel assertion plus an autotuning hang) while the dedicated build ran correctly (Section 5.3).

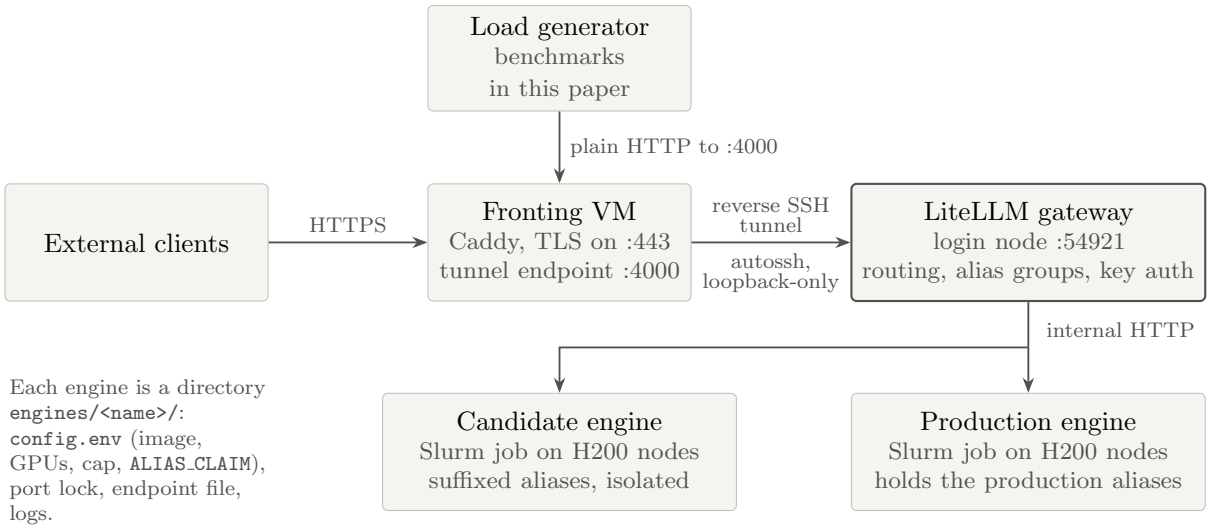


Figure 1: Request path from external clients through the gateway to engine jobs. A candidate engine registers only suffixed aliases, so it receives benchmark traffic without claiming production names; switching which engine holds the production aliases is a two-line `ALIAS_CLAIM` edit plus a gateway restart (Section 3). The benchmarks in this paper entered at the tunnel's plain-HTTP endpoint on the VM, so they exercised the tunnel, gateway, and engines but not the TLS front end.

Evaluation automation: skills and the agent. The A/B protocol of Section 4 is not a manual runbook. It is encoded as declarative, versioned procedures—*skills*, in the sense of composable packages of instructions and resources that agents load on demand [28]—which an LLM-based coding agent with shell access executes under human approval. Three skills carry the methodology. The *model-onboarding* skill governs how any new model enters the platform: it is official-documentation-first (the agent must locate the vendor's per-model Cookbook or Recipe pages and derive engine flags from them, rather than inventing configurations), recommends dual-framework deployment whenever both frameworks support a model, and encodes the image-version decision flow (versioned names, dedicated builds preferred, "once supported $\neq$ always supported"). The *concurrency-troubleshooting* skill encodes the layered diagnosis of Section 5.5 as a symptom→cause→fix table, and the *debug-journaling* skill requires that every failure and fix be recorded with its evidence. The repeated A/B measurements of Section 5 are driven by a versioned orchestration script (`benchmarks/repeats/`) that instantiates isolated engine copies, interleaves the bursts, and collects the evidence of every run. All procedures are included in the artifact repository, making the procedure itself a citable artifact rather than prose. Table 2 summarizes the division of labor: the agent performs the mechanical chain, while humans retain the decisions (which model to onboard, which framework releases to pin, and how to weigh verdicts).

4 Methodology

The evaluation asked two questions per model: (i) which framework serves it faster under a high-concurrency burst, and (ii) how enabling the checkpoint's speculative-decoding head changes the absolute throughput. We answer both with paired A/B measurements whose conditions are summarized in Table 3. The procedure was executed by the LLM-based agent described in Section 3; the human authors made the release decisions (which model, which framework versions to pin) and

Component	Implementation	Notes
Gateway	LiteLLM proxy, login node, port 54921	5.2% CPU / 333 MB (observed during the 1,000-user load test)
Authentication	Master key + per-user virtual keys (`key_tool.py`)	JSON key store with flock and atomic writes; no database; optional per-key RPM/TPM limits enforced in memory
Engine lifecycle	`start_models.sh` / `stop_models.sh` over Slurm	Idempotent dispatch, port locks, endpoint readiness files, two-stage validation (`validate_engine.sh`)
Routing config	`generate_runtime_config.py`	Static portal models + live probing of engine endpoints; alias groups with production/isolated claims
Traffic switching	`ALIAS_CLAIM` in each engine's `config.env`	Production alias set vs. suffixed aliases; switching frameworks = edit two lines + gateway restart (∼10 s), engines keep running
Container images	Versioned Apptainer SIF files (`sglang_0.5.20.sif`, `vllm_0.29.1rc1.sif`, dedicated `vllm_glm53-flash.sif`)	Version-named; the pull script refuses to overwrite an existing SIF unless forced; dedicated vendor builds used when a model requires them
External access	Caddy TLS + autossh reverse tunnel, plus SSH tunnel and web-proxy paths	Loopback-only tunnel binding; IP allowlists at the fronting VM

Table 1: Platform components.

accepted or rejected verdicts. Every engine flag is traceable to the official documentation the skill requires the agent to cite, and every run's statistics are machine-recorded.

Repeated-measurement protocol. For each model and round, both frameworks' engines were started as isolated copies (suffixed aliases, no production traffic) and kept online together, and each configuration was measured in five bursts. The bursts alternated between the two engines, and the leading framework was swapped on every repetition (SGLang first on odd, vLLM first on even), to reduce confounding by drift and run order; the observed difference between a framework's first- and second-position runs was at most about 5.2%, well below the framework gaps. Each engine received a warm-up of 64 concurrent requests (max_tokens 64) before measurement, excluded from the statistics, and consecutive bursts were separated by a 60 s cooldown. One exception: a dense vLLM repetition that was re-run on a fresh engine instance after a client-side failure did not receive the warm-up (Section 5.1). Two controls isolate confounds identified during the study: a cap-48 plain configuration for the dense model (Section 5.1) and a 600 s client deadline for the KDA R2 round (Section 5.3). All repeated measurements were taken on 2026-09-28/29; the original single runs of 2026-09-27, taken without warm-up, are reported alongside for comparison. Protocol deviations and their causes (for example, excluded nodes) are logged in the run manifest.

Fairness controls. Each pair of runs used (a) the same local checkpoint directory (bit-identical weights, no downloads), (b) the same GPU type, with both engines online in the same time window, (c) concurrency caps set equal on both sides, except where a framework cannot start at the shared cap (SGLang with EAGLE on the dense model; Section 5.1), and (d) engine startup parameters derived from each framework's official per-model documentation, with the experimental overrides recorded in the condition files (cap alignment, speculative-decoding settings, and SGLang's Mamba-cache size of cap $\times$ 5). Output tokens per R1 run stayed within $\pm 1.7\%$ of each model's mean across frameworks and repetitions (Table 3); R2 runs truncated by the client deadline are excluded from this

Pipeline stage	Agent-executed	Human decision
Locate official per-model documentation	✓	—
Scaffold + validate both frameworks' engines	✓	Framework releases to pin
Deploy under traffic-isolated aliases	✓	—
Run repeated paired benchmark	✓	Workload shape
Record verdict (validated-configuration registry)	✓	Interpretation
Switch production traffic / roll back	✓	Go/no-go
Failure diagnosis during any stage	✓ (from encoded symptom→cause procedures)	—

Table 2: Division of labor in the agent-assisted evaluation workflow. Skills and the orchestration script are version-controlled artifacts of the repository; re-running a decision still requires the same models, images, and hardware.

check, which in any case does not establish equal per-request work or equal output quality. Across all repeated runs, eleven requests were lost to transport-level connection resets about 1.5 s after submission, before any response; they count as not completed. The client harness, an httpx-based load generator (`stress_test.py`), drives a fixed pool of five short technical-QA prompts (cycled across requests at temperature 0.7) from the fronting VM through the reverse tunnel and the gateway to the engine, and records per-request latency, token counts, finish reason, and an output fingerprint. The client uses a 300 s request deadline, set earlier from the longest request observed on this platform (188 s) plus margin rather than from a service-level objective. The benchmark key carries no rate limit, and the gateway's upstream timeout (600 s) exceeds the client deadline.

Metrics. We report *aggregate* throughput (total output tokens divided by wall time) as mean ± sample standard deviation over repetitions, the per-repetition paired SGLang/vLLM throughput ratio, *completion rate* (requests completed within the client deadline ÷ total), and *tail latency* (P95). Because every burst releases all requests at once, P95 largely reflects queue-drain time rather than per-request service latency.

5 Results

5.1 Dense hybrid: SGLang +9.6%

On the 27B dense hybrid, SGLang led by 9.6% on average and in every paired repetition (Table 4); the paired SGLang/vLLM ratio was 1.097 ± 0.026 (range 1.071–1.139).

Enabling the checkpoint's integrated draft head cost both frameworks throughput, but for different reasons (Table 5). vLLM's MTP shares weights with the target model and kept cap 128; it lost 6.7%. SGLang's EAGLE loads draft weights plus an intermediate-state cache for the GDN blocks (13.8 GB at cap 48, 22.8 GB at cap 80), and with it SGLang cannot allocate KV cache for 128 concurrent requests on one H200: in our sweep, caps of 96, 112, and 128 failed at startup and 80 started (caps 81–95 were not tested); SGLang's own automatic setting is 48. The original study's configuration set cap 48 explicitly, with a Mamba cache of 240 (48×5); at that configuration SGLang lost 39.5% against its plain cap-128 configuration. A cap-48 plain control separates the

Condition	Dense comparison	Ultra-sparse MoE comparison	KDA comparison
Model (same checkpoint both frameworks)	Qwen3.8-27B (BF16, 52 GB)	Qwen3.8-Flash-Next (FP8, 173 GB; 180B/6B active)	GLM-5.3-Flash (FP8, 306 GB)
Architecture	Dense hybrid: GDN + full attention	Ultra-sparse MoE: 512 experts + 51B N-gram embedding	KDA linear attention + DSA, 288-expert MoE (8 active), 1M context
GPUs per engine	1× H200	4× H200 (TP4+EP4)	8× H200 (TP8+EP8 / TEP8)
Frameworks	SGLang 0.5.20 vs vLLM 0.29.1rc1	SGLang 0.5.20 vs vLLM 0.29.1rc1	SGLang 0.5.20 vs vLLM 0.28.1rc1 (dedicated GLM build)
Engine parameters	Official docs: SGLang Cookbook vs vLLM Recipe (per-architecture pages)	idem; `--max-num-seqs 256` per Recipe; SGLang cap aligned to 256; N-gram embedding on GPU for both	idem; BF16 KV cache (FP8 KV unsupported on Hopper for this model, per Recipe)
Concurrency cap	128 both sides (SGLang with EAGLE: 48; Section 5.1)	256 both sides	128 both sides
Workload	500 concurrent requests × max_tokens 500	1,000 × 800	1,000 × 800
Output tokens per R1 run	207k–209k	514k–528k	730k–735k
Repetitions	5 interleaved bursts per configuration	5	5, plus 3 further instance pairs (R1)

Table 3: A/B conditions. "R1" denotes the plain configuration (no speculative decoding); "R2" denotes speculative decoding enabled (the checkpoint's integrated MTP/NEXTN head, with each framework's recommended step/top-k settings). Architecture classes: `Qwen3_5ForConditionalGeneration` (dense), `Qwen4ExpForConditionalGeneration` (ultra-sparse MoE), `Glm5NextForConditionalGeneration` (KDA hybrid). TP/EP: tensor/expert parallelism; TEP: combined tensor–expert parallelism.

two effects: lowering the cap from 128 to 48 alone costs 47.3% (vLLM: 45.9%), while EAGLE at equal cap *raises* throughput by 14.8%. Even at the highest tested feasible cap (80), SGLang with EAGLE remained 34.1% below its plain configuration.

5.2 Ultra-sparse MoE: vLLM 2.40×

On the ultra-sparse MoE, the ranking reversed (Table 6): the paired SGLang/vLLM ratio was 0.417 ± 0.014, so vLLM delivered 2.40× the throughput of SGLang, and 2.02× that of SGLang's best configuration.

The same integrated draft head moved the two frameworks in opposite directions: SGLang's NEXTN gained 19.1%—consistent with the model's low 6B-active decode cost making draft-and-verify cheap—while vLLM's MTP lost 44.5%, matching the vendor recipe's warning against enabling it by default on this architecture. We attribute the R1 gap to MoE execution—vLLM's Triton MoE and FP8 pipeline for this architecture appeared more mature in the versions tested—although we

Metric	SGLang 0.5.20 (R1)	vLLM 0.29.1rc1 (R1)
Aggregate throughput ($n = 5$)	**3,882 ± 36 tok/s**	3,541 ± 95 tok/s
P95 latency	**50.9 ± 0.4 s**	55.9 ± 1.6 s
Completion rate	100% (all runs)	100% (all runs)
Original single run (2026-09-27)	3,823 tok/s	3,597 tok/s

Table 4: Dense comparison, R1 (plain). One vLLM repetition was re-run on a fresh engine instance after a client-side failure and, unlike the others, without the client warm-up; it measured 3,372 tok/s, 5.9% below the other four, and accounts for most of the vLLM spread. The rerun was not part of the original back-to-back pairing; excluding it, the other four pairs give vLLM $3{,}584 \pm 13$ tok/s and a paired ratio of 1.086 ± 0.011, and SGLang still led in every repetition.

Configuration	SGLang 0.5.20	vLLM 0.29.1rc1
R1 plain, cap 128	**3,882 ± 36**	3,541 ± 95
Plain, cap 48 (control)	2,046 ± 11	1,917 ± 24
R2 speculative, cap 48 (SGLang) / 128 (vLLM)	2,348 ± 10 (**−39.5%**)	3,304 ± 22 (**−6.7%**)
R2 speculative, cap 80 (highest tested feasible)	2,559 ± 14 (**−34.1%**)	—

Table 5: Dense comparison, speculative decoding and concurrency (tok/s, $n = 5$; percentages relative to the same framework's R1). SGLang uses EAGLE 3/1/4 (steps/draft/tokens); vLLM uses MTP×3. In the cap-48 plain control, the SGLang/vLLM ratio is 1.067 ± 0.015, the same direction as R1.

did not profile kernels to confirm the attribution.

5.3 KDA hybrid: vLLM +28%

On the KDA model, vLLM led (Table 7): the paired SGLang/vLLM ratio was 0.782 ± 0.037, and three further pairs of freshly started engine instances gave 0.757, 0.768, and 0.763 (all four pairs, weighted equally although the main pair has $n = 5$ and the others $n = 2$: 0.767 ± 0.010, i.e. vLLM +30%). All four pairs favored vLLM, and the cross-instance CV of the instance-mean throughput was 1.0% for SGLang and 2.0% for vLLM. The vLLM side requires a *dedicated* vendor build (0.28.1rc1 with precompiled GLM kernels); the generic nightly build failed during startup with a kernel assertion and a 31-minute autotuning hang, an observation that motivated our image-governance policy of preferring dedicated builds when they exist.

A single run reversed this ranking. The original single run, taken without warm-up, placed SGLang first by 28% and understated vLLM by 39% (2,857 vs. $4{,}719 \pm 201$ tok/s). Its engine log shows runtime compilation of TileLang kernels in the middle of the measured burst—nine compilation events logged by rank 0 in R1, and three per rank in R2—with generation throughput falling to 10–20 tok/s while they ran. In the repeated protocol, the runtime-JIT warnings that vLLM's monitor emits for this model appeared during the warm-up and never in a measured burst. A single-factor control, in which each engine started with an empty TileLang cache and ran one burst without warm-up followed by a second, confirmed that these runtime-JIT warnings occur only in the first burst, but did not isolate their throughput cost, and it did not reproduce the original cache state (with an empty cache, this build precompiles its TileLang kernels at startup): across three repetitions the first burst was not consistently slower, and one node showed generation stalls unrelated to JIT. We therefore attribute the original error to single-run measurement without

Configuration	SGLang 0.5.20	vLLM 0.29.1rc1
R1 aggregate throughput	3,299 ± 82	**7,921 ± 124**
R1 P95 latency	153.1 ± 3.8 s	**63.2 ± 1.1 s**
R2 (speculative)	3,930 ± 117 (NEXTN, **+19.1%**)[a]	4,394 ± 138 (MTP×3, **−44.5%**)
Best per framework	3,930 (NEXTN on)	**7,921 (plain)**
Original single run, R1	3,383	7,886

Table 6: Ultra-sparse MoE comparison (tok/s, $n = 5$). [a]One repetition completed 99.8% of requests (two connections reset about 1.5 s after submission, before any response); all other runs completed 100%. Both frameworks keep the 51B N-gram embedding on GPU; in the original single-run study, SGLang's default CPU offload of the embedding was 2.6% slower.

Configuration	SGLang 0.5.20	vLLM 0.28.1rc1 (dedicated)
R1 aggregate throughput	3,687 ± 71	**4,719 ± 201**
R1 P95 latency	193.5 ± 3.7 s	**150.3 ± 6.9 s**
R2 (speculative), 300 s client deadline	**3,362 ± 60** (**−8.8%**)	2,328 ± 108, censored[a]
R2 (speculative), 600 s client deadline	**3,251 ± 68**	2,420 ± 11[b] (**−48.7%**)
Best per framework	3,687 (plain)	**4,719 (plain)**
Original single run, R1	3,669	2,857

Table 7: KDA comparison (tok/s, $n = 5$; percentages relative to the same framework's R1). R2 uses SGLang EAGLE 5/1/6 and vLLM MTP×5; the two R2 rounds ran on different engine instances. [a]0–10.5% of requests per repetition were not completed within the 300 s deadline, so throughput counts completed requests only. [b]No request reached the deadline; 4,997 of 5,000 requests completed, the other three lost in one repetition to connection resets about 1.5 s after submission.

warm-up—runtime compilation and instance-level stalls both distort a single burst—rather than to JIT alone.

Speculative decoding and the client deadline. Speculative decoding lowered throughput on both frameworks. On vLLM it also pushed the burst's queue-drain time past the client's 300 s deadline: 0–10.5% of requests were abandoned by the client in individual repetitions, although the engine reported no errors. With a 600 s deadline, no request timed out (the slowest took 305 s); 4,997 of 5,000 requests completed, the other three lost to early transport-level connection resets, and vLLM's R2 throughput, no longer deadline-censored, was $2{,}420 \pm 11$ tok/s, 48.7% below its plain configuration. The operational consequence was immediate: our production SGLang deployment had been running with MTP enabled, and disabling it—a change shipped the same day by editing the engine's `config.env`—raises throughput by 9.7% in the repeated measurements. Under this workload vLLM would serve GLM-5.3-Flash 28% faster still; production has not been switched.

5.4 Cross-architecture synthesis

Table 8 and Figure 2 collect the headline results: the leading framework depended on the architecture, and the margin ranged from 9.6% to 2.40×.

Two regularities stand out. First (Figure 2a), the framework gap was smallest on the well-trodden dense hybrid (9.6%) and largest on the newer ultra-sparse MoE (2.4×), which may reflect how recently each framework acquired mature kernels for a given architecture—though with one model per architecture we cannot separate this from framework-version and workload differences. Second (Figure 2b), speculative decoding lowered throughput in five of the six framework–model pairs in

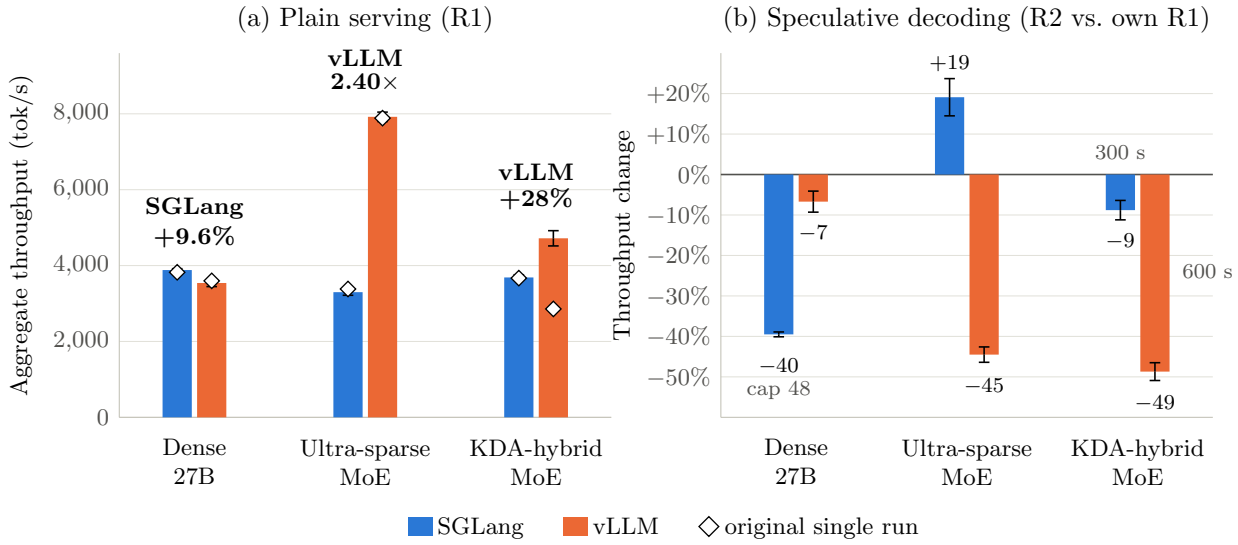


Figure 2: The three comparisons at a glance (values from Tables 4–7). (a) Plain-serving throughput, mean ± SD over five repetitions; the label above each pair names the leader and its margin, and open diamonds mark the original single runs, of which the KDA vLLM run understated throughput by 39% and reversed the ranking. (b) Change in aggregate throughput when the checkpoint's speculative-decoding head is enabled, as the ratio of mean throughputs $r = \bar{m}_{R2}/\bar{m}_{R1}$ minus one; error bars propagate the two standard deviations, $100\,r\sqrt{(s_{R2}/\bar{m}_{R2})^2 + (s_{R1}/\bar{m}_{R1})^2}$, ignoring covariance, and are not standard errors or confidence intervals. Five of six bars are below zero. SGLang's dense bar is at its EAGLE cap of 48 (Section 5.1); for KDA, SGLang is from the 300 s round and vLLM from the 600 s round, on different engine instances (Section 5.3).

their deployable configurations. For vLLM on all three models and SGLang on the KDA model, the loss is consistent with draft-and-verify work competing with decode batches that already occupy the GPU, although we did not profile it; for SGLang on the dense model it is a memory effect (Section 5.1); and the single gain (SGLang NEXTN on the 6B-active MoE) is consistent with cheap decodes leaving headroom for drafting.

5.5 Platform-level capacity and reliability

Beyond per-engine comparisons, the platform sustained a mixed workload of 1,500 concurrent users across all three engines with 100% success, 8,146 tok/s aggregate throughput, and 76.8 s P95 from the fronting VM through the tunnel and gateway (Table 9). Gateway overhead at this load was 2.5% CPU and 405 MB of memory, and no gateway-related failures were observed at the tested burst size. The measurements in this section are single runs, and the gateway CPU and memory figures here and in Table 1 are observations recorded in the operations log at the time rather than retained telemetry.

We report this debugging arc as a result in its own right for two reasons. First, an initial hypothesis—that an 8% ReadError rate indicated a single-flow ceiling of the reverse tunnel—was *refuted* by the fix-and-remeasure cycle: after correcting both fd limits, 1,500 concurrent users traversed the single tunnel flow at 100% success, so no such ceiling was reached below 1,500. Second, the diagnosis illustrates a methodology we now apply routinely: measure each layer independently (client → tunnel → gateway → engine), fix one layer at a time, and re-run the identical workload after

Architecture (model)	R1 leader	Margin ($n = 5$)	R2 vs. own R1	
			SGLang	vLLM
Dense hybrid (Qwen3.8-27B)	SGLang	+9.6%	−39.5%[a]	−6.7%
Ultra-sparse MoE (Qwen3.8-Flash-Next)	vLLM	2.40×	+19.1%	−44.5%
KDA-hybrid MoE (GLM-5.3-Flash)	vLLM	+28%	−8.8%[b]	−48.7%[c]

Table 8: Synthesis across the three comparisons. [a]At SGLang's EAGLE cap of 48; at equal cap EAGLE gains 14.8%, and at the highest tested feasible cap (80) the loss is 34.1% (Section 5.1). [b]300 s round, in which no SGLang request approached the deadline. [c]600 s round, which removes the truncation of the 300 s round (Section 5.3). Each framework uses its documented speculative settings; the five-negative, one-positive pattern holds for either dense SGLang cap (48 or 80) and either KDA deadline round. Framework versions as in Table 3.

Stage of the reliability investigation	Success	Aggregate	P95	Root cause identified
Initial mixed run (1,500 users)	67.9%	—	—	Client shell `ulimit -n` 1024 → fd exhaustion (Errno 24) at ∼1,020 open sockets
After client fix, via tunnel	91.8%	8,251 tok/s	70.4 s	Per-connection sshd on the tunnel VM also capped at 1024 fds (verified via `/proc/<pid>/limits`)
Bypassing the tunnel (direct)	100%	8,156 tok/s	77.6 s	Isolated remaining losses to the VM↔gateway segment
After `/etc/security/limits.conf` fix, via tunnel	**100%**	8,146 tok/s	76.8 s	Correct fix: PAM `limits.conf` (systemd override alone does not apply to per-connection sshd); tunnel re-established after the fix

Table 9: Layered attribution of mixed-load failures (1,500 users × 500 tokens; three engines live).

each fix rather than trusting bypass experiments to localize faults finer than segment granularity.

6 Discussion

Framework choice is a per-model decision, and it is large. The observed spread—from a 9.6% margin to a 2.40× gap—is the largest lever we measured on correctly configured engines. Concurrency-cap tuning, in separate single-run measurements on the same engines, raised aggregate throughput by 29% (MoE, cap 48 → 100) and 32% (KDA, cap 64 → 128), and restoring a misconfigured SGLang baseline on the dense engine had earlier been worth +48%. In operational terms, the measured framework gap on the ultra-sparse MoE exceeded the effects of every configuration change we evaluated. The practical implication we adopted institutionally is a *dual-framework onboarding policy*: every new model is deployed on both frameworks under suffixed aliases, A/B-tested under the production workload, and promoted by a two-line configuration change; the platform of Section 3 makes this a routine procedure rather than a redeployment project. The results do not favor either framework in general: each led on at least one of the three architectures.

Single-run benchmarks can pick the wrong champion. Our first comparison of the KDA model, a single burst without warm-up, named SGLang the winner by 28%; five warmed-up, interleaved repetitions and three further engine-instance pairs named vLLM the winner by 28–30%, with all four instance pairs favoring vLLM. The single run used the same checkpoint, hardware, and documented parameters as the repeated protocol; what it lacked was warm-up and repetition, and

on a new architecture whose kernels compile at runtime, one burst can absorb compilation stalls that a steady-state service never sees. Because the orchestration is a versioned artifact, the correction was produced by re-running the procedure rather than by re-engineering the study; we recommend that framework comparisons on new architectures report repeated, warmed-up measurements as a minimum.

Speculative decoding at high concurrency. Our results add same-checkpoint, repeated evidence from a production platform on how speculative decoding behaves under load: with 500–1,000 concurrent requests, enabling the integrated draft head lowered net throughput in five of six pairs. For vLLM on all three models and SGLang on the KDA model this is consistent with the mechanism goodput analyses describe—verification work competing with useful decoding when speculation ignores load [8]—although we did not profile it; for SGLang on the dense model the loss is a memory effect (below). Adaptive systems such as TurboSpec address this by gating speculation on measured goodput [8], and AdaServe by customizing it to per-request SLOs [9]; our measurements suggest a cruder but zero-engineering-cost policy—disable integrated MTP/NEXTN by default for short-prompt, high-concurrency throughput serving of these architectures, and enable it only where a measurement shows a gain, as for SGLang NEXTN on the MoE. We do *not* conclude that speculative decoding is useless: at low concurrency, for single-stream latency, or—per MagicDec's analysis—for long contexts under high batch [29] it can remain appropriate, and the one positive case here (SGLang NEXTN, +19% at 6B active parameters) is consistent with a model-dependent crossover, as in the goodput formulation [8]. The decision should be made from a measurement at the deployment's own concurrency, not from the feature's reputation.

Memory accounting can decide speculative feasibility. On the dense hybrid, the decisive cost of SGLang's EAGLE was not computation but memory: the draft weights and the GDN intermediate-state cache (13.8 GB at cap 48, 22.8 GB at cap 80) made 80 the highest feasible cap we tested on one H200, and the concurrency lost (−47% from 128 to 48) outweighs what drafting gains (+15% at equal cap). vLLM's MTP shares the head weights within the checkpoint and preserved the cap. On models whose caches dominate memory, this implementation difference can outweigh acceptance-rate considerations, and it is invisible in feature lists; a feasibility sweep of the concurrency cap is a cheap way to expose it.

Dedicated builds and version governance. The KDA comparison could not be run on the generic vLLM nightly at all (kernel assertion; autotuning hang), while the vendor's dedicated build ran correctly. Our resulting policy—version-named images that are not overwritten by default, dedicated builds preferred when they exist, "once supported $\neq$ always supported"—converted what could have been days of kernel-level debugging into a one-line image selection. Framework *version* is a first-class experimental variable on new architectures, and evaluations that do not pin and report it are hard to reproduce.

What the platform measurements do and do not show. The layered analysis of Section 5.5 observed no gateway- or tunnel-related failures at the tested burst size: the gateway used 2.5% CPU at 1,500 users, and no single-flow tunnel ceiling was observed below 1,500 users. These runs did not pass through the TLS front end and include no capacity sweep, so they do not establish where the next bottleneck lies.

Agents as experimentalists. Beyond its findings, this study is a working example of an LLM-based agent assisting with an experimental procedure under human approval. Autonomous research agents have been demonstrated for closed-loop scientific discovery [30]; agent-driven simulation has been proposed to track the fast-moving serving design space because real deployments are costly [31]; and AutoTuneBench has agent systems auto-tune both engines under a strict measurement protocol [26]. Our pipeline is closest to the last of these but different in object: AutoTuneBench tunes engine configurations, whereas our skills drive the full production lifecycle—deployment behind

a live gateway, benchmarking through the external traffic path, verdict recording, and production traffic switching—on infrastructure that keeps serving users throughout.

What made the automation trustworthy was not the agent's raw capability but the enclosure: declarative skills that require documentation-traceable configuration, traffic isolation that makes candidate engines harmless to production, versioned orchestration that records every run, and mandatory evidence journaling. Within that enclosure, the agent performed work that would otherwise gate on scarce systems-engineering time—including the repeated measurements and controls of Section 5, which the agent planned, executed, and diagnosed (excluding faulty nodes and correcting a mis-specified condition along the way) under the first author's direction. Related benchmarks evaluate how well agents do DevOps tasks [32]; here the DevOps task itself—deploying, benchmarking, and switching production inference infrastructure—was the experiment. We take this as an operational answer to the question this paper opened with: "which framework should serve this model?" is better answered by a repeated measurement whose procedure is a versioned artifact of the repository than by folklore or release notes.

7 Limitations and Future Work

Five limitations bound our conclusions. *Workload.* All comparisons use one workload shape—a single burst of short technical-QA prompts at uniform max_tokens, released at once—so latency percentiles reflect queue drain rather than steady-state service, and results may differ for long contexts, open-loop arrivals, or streaming metrics (time to first token and time per output token), none of which we measured. *Mechanism.* We did not profile kernels, GPU utilization, or speculative acceptance lengths, so the attributions offered for the MoE gap and for the speculative-decoding losses remain hypotheses; the harness records output fingerprints but runs no output-quality evaluation. *Scope.* We test one model per architecture, one GPU type, and single-engine deployments; the MoE gap in particular should be expected to move as kernels mature, the speculative cap sweep did not test caps 81–95, and multi-instance configurations were validated only for success rate (Section 5.5). We report an agent-assisted deployment workflow, not a controlled comparison of agent and human effectiveness. *Variance.* Repetitions within an engine instance agreed closely (CV mostly below 3%), but one dense repetition without warm-up measured 5.9% below the others; in a KDA control without warm-up, first bursts measured 3,839–4,170 tok/s and later bursts 2,999–4,622 tok/s, the lowest on one node with generation stalls unrelated to JIT; separately, the four warmed-up instance means averaged 4,866$\pm$98 tok/s; the concurrency-tuning and reliability numbers of Sections 5.5 and 6 are single runs. *Provenance.* SHA-256 digests of the container images were recorded for the repeated measurements; the original single runs used images that were later renamed to version-named files, and no digests were recorded for them. Model checkpoints are identified by file listing rather than by revision.

Future work should extend the matrix in five directions: mixed and open-loop workload shapes with streaming TTFT/TPOT; FP8-versus-BF16 and KV-cache-dtype axes, which our Hopper hardware partially constrains for the KDA model (FP8 KV unsupported); later framework releases, to calibrate how quickly such evaluations expire; a concurrency sweep for speculative decoding, recording acceptance lengths; and kernel-level profiling to test the mechanism attributions offered in Section 5.

8 Conclusion

We built a multi-model API service on an HPC cluster—a LiteLLM gateway over Slurm-managed engines on H200 GPUs—and, confronted with the recurring question of which serving framework to assign to each new model, encoded the comparison as versioned skills and an orchestration script that an LLM-based agent executes under human approval: scaffold both frameworks from official documentation, deploy them under traffic-isolated aliases, benchmark both in repeated, interleaved, warmed-up bursts, record the verdict, and switch production traffic when warranted. Applied to three architecturally distinct LLMs, the comparisons found that the leading framework depended on the architecture—SGLang +9.6% on a dense hybrid, vLLM 2.40× on an ultra-sparse MoE, vLLM +28% on a KDA-hybrid MoE—and that speculative decoding with integrated draft heads lowered throughput in five of six framework–model pairs, in one case because of memory rather than computation. A single run without warm-up had named the wrong winner for the KDA model. Together, the results argue for treating framework selection and speculative decoding not as defaults to be inherited but as per-model, per-workload decisions—and for making the repeated measurement that informs them cheap and safe enough to run on every new model, which is the purpose of the agent-assisted procedure described here.

AI-Use Disclosure

LLM-based coding agents assisted with engine deployment, benchmark orchestration, failure diagnosis, literature searches and reference checks, and manuscript drafting, review, and revision, including figures and LaTeX preparation. The agents operated under human direction using the versioned procedures described in this paper. The tools used were opencode (GLM-5.3), Claude Code (Anthropic Claude Opus 5.5), and GPT-6 (ASTRA). The human authors defined the research questions and methodology, made deployment and release decisions, and retain responsibility for the results, references, and final manuscript.

Author Contributions

Chao-Chun Chuang: Conceptualization, Methodology, Software, Investigation, Formal analysis, and Writing — original draft (AI-assisted, under the author's direction; see AI-Use Disclosure). **Po-Hsiang Lin:** Investigation, Data curation, Validation, and Writing — review and editing. **Liang-Hsun Huang:** Validation (checking the manuscript's descriptions of the models' architectures, parameters, and terminology against their official documentation) and Writing — review and editing.

Funding

This work was supported in part by the National Science and Technology Council (NSTC), Taiwan, under Grant Nos. NSTC 115-2410-H-A49-046-MY3 and NSTC 114-2634-F-006-002.

Competing Interests

The authors declare that they have no competing interests.

Acknowledgments

The authors gratefully acknowledge the National Center for High-performance Computing (NCHC), National Institutes of Applied Research (NIAR), Taiwan, for providing the research resources, computational infrastructure, and platform services that supported this work.

Data and Artifact Availability

The artifact repository (release `v1.0`)

`https://github.com/gemini960114/no-permanent-champion-research-artifact`

provides a curated snapshot of the experimental materials supporting this paper, including raw measurements, analysis scripts, benchmark configurations, and redacted execution logs. The repeated measurements are in `benchmarks/results/repeats/20260928-153829/`: per-run summaries and per-request records for every configuration and control, the run manifest and event log, the redacted engine configurations and logs (including, in `original_run_logs/`, the engine logs of the original single runs cited in Section 5.3), Slurm job records, and the aggregated tables; the original single runs are in `benchmarks/results/`, and the client-side summary of the initial 67.9% mixed run (Table 9) is in `benchmarks/results/client_vm/`. The repository also contains the client harness (`benchmarks/stress_test.py`), the orchestration, probe, and aggregation scripts with their condition files (`benchmarks/repeats/`), the validated engine configurations (`engines/KNOWN_GOOD.md`), and the evaluation skills (`.agents/skills/`); `analysis/reproduce_paper_numbers.py` recomputes a documented selection of the paper's numbers from the raw data; the gateway CPU and memory figures are recorded observations without raw telemetry. Host names, network addresses, account identifiers, and file-system paths are replaced by placeholders, with compute nodes mapped consistently so that placements and event timelines remain traceable. The repository supports result verification and re-analysis, not full deployment reproduction: it does not include the platform's gateway, engine-lifecycle, and routing code, the model checkpoints, or the container images, and the orchestration scripts are provided as a record of the procedure rather than as a turnkey deployment. It does not include the internal platform's development history or the agent-session logs, which were not curated as a research record and contain unredacted deployment details. The manuscript-writing skills are third-party skills pinned by content hash in `skills-lock.json` but not redistributed.

References

[1] Forough Majidi, Mohammad Mehdi Morovati, Foutse Khomh, and Heng Li. LLM serving in the wild: An empirical study of frameworks, methods, and system designs, 2026. arXiv:2608.03036.

[2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.

[3] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2312.07104.

[4] Saicharan Kolluru. Comparative analysis of large language model inference serving systems: A performance study of vLLM and HuggingFace TGI, 2025. arXiv:2511.17593.

[5] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. LLM inference serving: Survey of recent advances and opportunities, 2024. arXiv:2407.12391.

[6] David Pape, Jonathan Evertz, and Lea Schönherr. The silent hyperparameter: Quantifying the impact of inference backends on LLM reproducibility, 2026. arXiv:2605.19537.

[7] Ranran Haoran Zhang, Aysa Xuemo Fan, David Munhá Correia, et al. SiliconBench: Speed, memory, and fidelity for LLM serving on unified-memory desktops, 2026. arXiv:2609.19169.

[8] Xiaoxuan Liu, Jongseok Park, Langxiang Hu, Woosuk Kwon, et al. TurboSpec: Closed-loop speculation control system for optimizing LLM serving goodput, 2024. arXiv:2406.14066.

[9] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, et al. AdaServe: Accelerating multi-SLO LLM serving with SLO-customized speculative decoding, 2025. arXiv:2501.12162.

[10] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[11] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills, 2023. arXiv:2308.16369.

[12] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. arXiv:2401.09670.

[13] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving, 2025. arXiv:2501.01005.

[14] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report, 2025. arXiv:2505.09388.

[15] DeepSeek-AI. DeepSeek-V3 technical report, 2024. arXiv:2412.19437.

[16] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model, 2024. arXiv:2405.04434.

[17] GLM-5 Team, Aohan Zeng, et al. GLM-5: from vibe coding to agentic engineering, 2026. arXiv:2602.15763.

[18] Kimi Team, Yu Zhang, Zongyu Lin, et al. Kimi Linear: An expressive, efficient attention architecture, 2025. arXiv:2510.26692.

[19] Kimi Team, Yifan Bai, Yiping Bao, et al. Kimi K2: Open agentic intelligence, 2025. arXiv:2507.20534.

[20] Yanqi Yu, Pingwei Sun, Jianchao Tan, et al. DASC: Decay-aware state compression for hybrid linear-attention serving, 2026. arXiv:2608.30386.

[21] Tao Zhang, Jianchao Tan, Pingwei Sun, et al. DAMP: Decay-aware mixed-precision recurrent-state quantization, 2026. arXiv:2608.27513.

[22] SLAM Labs, Oleksiy Ostapenko, et al. Super apriel: One checkpoint, many speeds, 2026. arXiv:2604.19877.

[23] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.17192.

[24] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.10774.

[25] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.15077.

[26] Li Chen. AutoTuneBench: Trustworthy measurement for agent auto-tuning of LLM serving engines, 2026. arXiv:2609.18123.

[27] Keisuke Kamahori, Shihang Li, Simon Peter, and Baris Kasikci. VibeServe: Can AI agents build bespoke LLM serving systems?, 2026. arXiv:2605.06068.

[28] Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward, 2026. arXiv:2602.12430.

[29] Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen, et al. MagicDec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding, 2024. arXiv:2408.11049.

[30] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery, 2024. arXiv:2408.06292.

[31] Wonung Kim, Hyunmin Choi, Minsu Kim, Jaehong Cho, et al. Simthesizer: An agent-driven simulation framework for LLM serving systems, 2026. arXiv:2608.24650.

[32] Yuheng Tang, Kaijie Zhu, Bonan Ruan, Chuqi Zhang, et al. DevOps-Gym: Benchmarking AI agents in software DevOps cycle, 2026. arXiv:2601.20882.